

Application Support Case Study

Troubleshooting a Full-Stack Healthcare Scheduling SaaS

Clayton Parnell | Software Engineering B.S. | Application Specialist / Application Support Candidate

Purpose

This document reframes my full-stack software project as an application-support and business-systems case study. It is designed for hiring managers evaluating Application Specialist, Application Support Analyst, Business Applications Support, Technical Support Analyst, Product Support, or Implementation Support candidates.

What this demonstrates: Structured triage, issue reproduction, workflow analysis, data-integrity reasoning, backend/API validation, user-facing communication, regression awareness, and documentation discipline.

Project scope: A portfolio-grade healthcare scheduling SaaS using React, TypeScript, Django REST Framework, PostgreSQL, JWT authentication, Docker, and AWS deployment patterns.

Privacy note: All patient/provider data is synthetic and non-PHI. The system models healthcare-style scheduling complexity without storing real medical data.

GitHub source: github.com/cparn8/healthcare-saas

1. Executive Summary

This case study documents how I approach application-support problems using a full-stack scheduling system that models real operational constraints: providers, patients, locations, business hours, appointment types, block times, overlapping appointments, authentication, role-based permissions, and deterministic demo data.

The goal is not to present the project as a deployed commercial medical product. The goal is to show the kind of systems thinking required to support business applications: identifying user impact, reproducing symptoms, isolating likely causes, validating backend/API behavior, checking data visibility and permissions, documenting findings, and preventing regressions.

Area	System Behavior	Application Support Value
Scheduling workflows	Day/week schedule views, appointment creation/editing, block times, recurrence metadata, overlap confirmation.	Demonstrates ability to reason through user-facing business workflows and edge cases.
Business rules	Conflict detection, location-specific hours, patient requirements, explicit overlap override.	Shows comfort with rule-driven application behavior and backend validation.
Identity and access	JWT authentication, provider/admin distinction, object-level authorization.	Maps to common support issues involving login, permissions, and access scope.
Data reliability	PostgreSQL as source of truth, deterministic seeding, transactional writes.	Supports disciplined troubleshooting of missing, inconsistent, or invalid records.
QA discipline	Functional, negative, regression, API, and permission tests documented across the project.	Shows readiness to validate fixes and prevent recurrence after changes.

2. Support Framework Used in This Case Study

For each scenario, I use a consistent support workflow rather than jumping directly to code or assumptions.

1. Clarify the reported symptom and user impact.
2. Reproduce the behavior using the same workflow, filters, date, role, and location context.
3. Separate UI display issues from API/data issues by checking the request, response, and persisted record.
4. Check permissions, account context, and role boundaries when behavior differs between users.
5. Trace business rules: provider, date, location, time range, appointment type, block status, business hours, conflict rules, and recurrence metadata.
6. Identify likely root cause and determine whether the issue is configuration, data, validation, UI rendering, API behavior, or user misunderstanding.
7. Communicate the finding in plain language, provide a workaround if needed, and document reproduction/resolution steps.
8. Add or update regression coverage so the same issue is less likely to reappear.

Support principle

The frontend is treated as a presentation layer. The backend/API and database are treated as the authority for scheduling rules, permissions, and persisted state.

3. System Context

The project models a business-critical scheduling application where small errors can create visible operational problems: incorrect availability, accidental double-booking, missing appointments, invalid block times, or users seeing records they should not be able to modify.

Concept	Meaning in the system
Provider	A scheduling user linked to a Django account and scoped to provider-owned appointment data.
Location / Office	A stable scheduling identifier with location-specific business hours and office-scoped conflict rules.
Appointment	A unified scheduled entity representing either a patient appointment or a block such as lunch/admin/out-of-office.
Conflict	An overlap involving the same provider, same date, same office, and overlapping time window.
Override	An intentional retry with <code>allow_overlap=true</code> after the backend returns a conflict response.
Business hours	Location-specific hours that shape normal scheduling behavior and support predictable UI rendering.
Demo data	Synthetic, deterministic, repeatable data used for QA, screenshots, demos, and regression validation.

4. Scenario 1: User Reports That an Appointment “Disappeared”

Reported symptom: A user believes an appointment was created successfully, but it is not visible in the schedule grid.

Initial support risk: The issue could be a true data-loss problem, a filter/date-range issue, a pagination problem, a deduplication/rendering defect, or a misunderstanding of the selected provider/location context.

Investigation Path

- Confirm the exact date, provider, office/location, appointment type, status, and view mode used by the user.
- Check whether the appointment exists in the backend response for the requested date range.
- Verify that frontend filters are not hiding the appointment by provider, office, type, status, or blocked/appointment category.
- Check whether the visible-range logic is excluding the appointment because of date parsing, week boundary, or time normalization behavior.
- Verify deduplication behavior. Appointments should be deduplicated by appointment ID, not by matching time slot, because legitimate double-bookings can share time ranges.

- Inspect pagination/list loading behavior. If the API returns paginated data, the frontend must fetch all relevant pages or the UI can appear to lose records.
- Confirm whether the appointment is a block, patient appointment, recurring metadata record, or status-filtered item.
- Document the root cause and add regression checks for the exact visibility failure mode.

Root Cause Patterns This Scenario Tests

Possible Cause	How to Validate	Resolution Pattern
Record exists but is hidden by filter	Compare API response with active provider/location/status/type filters.	Clarify filter behavior or adjust filter defaults/labels.
Record exists but not in current visible range	Check date window and client-side visible-range logic.	Fix date normalization or range calculation; add regression scenario.
Record exists but paginated out	Inspect API pagination and frontend list loading.	Fetch all required pages or explicitly scope requests to the visible range.
Double-booked record hidden by dedupe logic	Compare records with different IDs but overlapping times.	Deduplicate by ID only; preserve legitimate overlaps.
Recurring metadata not materialized	Check whether the appointment is a recurrence definition rather than an instance.	Explain current behavior and document future recurrence expansion needs.

Support takeaway

A “missing appointment” report should not be treated as one generic bug. The support path should isolate whether the appointment is missing from the database, missing from the API response, excluded by permissions, filtered out by UI state, or rendered incorrectly.

5. Scenario 2: User Cannot Create or Save an Appointment

Reported symptom: The user fills out the New Appointment or Edit Appointment form, but the system rejects the save or asks for confirmation.

Initial support risk: The save failure may be correct behavior. In a scheduling system, a rejection can indicate that the system is protecting business rules rather than malfunctioning.

Investigation Path

- Reproduce the exact create/edit workflow and collect the validation message returned by the backend.
- Check required fields: provider, date, office/location, start time, end time, appointment type, and patient requirement for non-block appointments.
- Validate time ordering: end time must be after start time.
- Check business hours for the selected location and date.
- Check whether the appointment overlaps another appointment or block for the same provider/date/location.
- If there is a conflict, confirm that the backend returns an explicit conflict response and that the UI asks for intentional confirmation before retrying with `allow_overlap=true`.
- Check user role and provider ownership if the appointment belongs to another provider.

- Document whether the system is behaving correctly, whether the user needs guidance, or whether a real defect exists.

Validation Area	Expected Behavior	Support Interpretation
Conflict detection	Overlap rejected by default; explicit confirmation required for override.	A rejection may be correct. Confirm user intent before allowing double-booking.
Patient requirement	Patient required for non-block appointments; blocks do not require patients.	Clarify appointment type choice and field requirements.
Business hours	Location-specific hours constrain normal scheduling behavior.	Verify location and date context before diagnosing as a bug.
Authorization	Providers cannot modify records outside their allowed scope unless admin.	401/403 indicates access/session/permission issue, not form failure.
Payload validity	Malformed or incomplete data rejected with explicit error response.	Use error response to identify missing or invalid fields.

Support takeaway

The correct support posture is not “make the save work.” It is “determine whether the rejection protects a valid business rule, then explain the rule or fix the defect.”

6. Scenario 3: Location, Business-Hours, or Recurrence Confusion

Reported symptom: A user reports that availability, appointment placement, or recurring behavior does not match expectations.

Initial support risk: The symptom may involve location-specific rules, office-scoped conflicts, closed days, time normalization, or intentionally non-materialized recurrence metadata.

Investigation Path

- Confirm the selected location/office and whether the provider is scheduled there on the reported date.
- Check location-specific business hours and closed-day rules.
- Validate whether the appointment is scoped to the correct office slug.
- Confirm whether conflicts are being evaluated within the same provider/date/office context.
- For recurring appointments, inspect recurrence metadata: selected weekdays, interval, end condition, and date parsing.
- Distinguish between recurrence metadata and materialized future appointment rows. The current design stores recurrence metadata rather than expanding all future instances.
- Document whether the behavior is intended, a known limitation, or a candidate for future enhancement.

Topic	Design Reality	How Support Should Handle It
Office-scoped conflicts	Conflicts depend on provider, date, office, and overlapping time window.	Check office/location before assuming a conflict or non-conflict is wrong.

Business hours	Hours are location-specific and influence schedule rendering/selection.	Verify the selected office and date; compare UI behavior with backend configuration.
Blocks	Blocks are first-class appointments and participate in conflict detection.	Explain that lunch/admin/out-of-office blocks can intentionally prevent normal scheduling.
Recurrence	Recurring appointments are stored as metadata, not expanded into child rows.	Set expectations and document as a known future-work area if full visualization is requested.

7. Scenario 4: Login, Session, or Permission Issue

Reported symptom: A user cannot access the app, loses session state, or cannot perform an operation that another user can perform.

Initial support risk: Authentication failures, token expiry, missing provider linkage, role boundaries, and object-level permissions can look similar from the user's perspective.

Investigation Path

- Determine whether the issue is authentication (who are you?) or authorization (what are you allowed to do?).
- Check for missing, expired, or invalid JWT access token behavior.
- Confirm that the account has a linked Provider record when the user is expected to schedule.
- Compare the action against role boundaries: provider vs admin.
- Check whether the attempted record belongs to the authenticated provider or requires elevated access.
- Interpret 401 as authentication/session failure and 403 as permission failure.
- Document the resolution in language the user can understand without exposing internal security details.

Support takeaway

Permission issues should be diagnosed with precision. A secure system should fail visibly and consistently, but support must explain the difference between session failure, role restriction, and object ownership.

8. QA and Regression Discipline

The project includes formal QA documentation covering test strategy, negative testing, regression scenarios, risk analysis, traceability, smoke testing, release readiness, automated test architecture, and CI execution. The most relevant support lesson is that fixes should not end when the immediate symptom disappears.

QA Artifact	What It Covers	Application Support Value
Test strategy	Manual and automated testing across auth, scheduling, permissions, and business-critical workflows.	Shows ability to validate quality beyond ad-hoc clicking.
Negative tests	Missing auth, invalid tokens, invalid payloads, overlap rejection, location integrity failures.	Demonstrates fail-safe thinking and prevention of invalid states.

Regression scenarios	Login, token refresh, appointment creation/editing, overlap confirmation, day/week rendering, location constraints.	Protects core workflows after fixes or refactors.
Risk analysis	Double booking, date drift, demo corruption and corresponding mitigations.	Shows risk-based prioritization instead of equal-weight testing.
Known limitations	Single-tenancy, limited UI automation, no audit log, non-materialized recurrence.	Shows honest documentation and scope discipline.

Regression mindset

A support fix should answer three questions: What changed? What user workflow did it affect? What regression check prevents the same issue from returning?

9. How This Maps to Application Specialist / Application Support Work

Role Need	Case Study Evidence	Transferable Value
Triage ambiguous reports	Missing appointment, rejected save, recurrence/location confusion, access issue scenarios.	Can turn vague complaints into reproducible support paths.
Understand workflows	Scheduling rules across providers, offices, hours, blocks, patients, and statuses.	Can learn business processes rather than treating every issue as a generic ticket.
Bridge users and technical teams	Plain-language symptom framing plus backend/API/data validation steps.	Can communicate with both end users and developers/vendors.
Protect data integrity	Backend-enforced rules, explicit overlap overrides, referential safeguards, permission checks.	Understands why some “errors” are correct system behavior.
Document and prevent recurrence	QA artifacts, regression scenarios, negative tests, known limitations, future-work notes.	Can maintain knowledge base and support change management.
Support production-like systems	Dockerized environments, CI, AWS deployment architecture, JWT auth, RDS/PostgreSQL.	Has exposure to real SaaS patterns beyond a simple frontend demo.

10. Technical Appendix

The project is intentionally full-stack so troubleshooting can be reasoned through across UI, API, authentication, business rules, and persisted data.

Layer	Technologies / Patterns
Frontend	React, TypeScript, Tailwind CSS, Axios, React Router, custom hooks, deterministic schedule rendering.
Backend	Python, Django, Django REST Framework, SimpleJWT, serializers, viewsets, permission checks, business-rule validation.
Database	PostgreSQL relational schema with persisted appointment, provider, patient, location, and settings data.
Runtime / Deployment	Docker, Docker Compose, Gunicorn, Nginx, AWS ECS Fargate, RDS, ALB, Route 53, ACM.
QA / CI	pytest, pytest-django, Dockerized test execution, GitHub Actions, manual QA documentation, regression checklist.
Privacy / Demo Safety	Synthetic non-PHI data, deterministic reset command, non-routable/fake identifiers, documented security limitations.

11. Closing Summary

This case study is meant to show how I think when supporting applications: start with the user report, reproduce the workflow, isolate the technical layer, validate rules and permissions, communicate clearly, and preserve reliability through documentation and regression coverage. The project is software-engineering-heavy by construction, but the support value is broader: it demonstrates practical reasoning about business applications, user workflows, data integrity, access control, troubleshooting, and maintainability.